

Projet P12 : Détectez de faux-billets avec Python

Détection de Faux Billets par Machine Learning

Développement d'un modèle de classification binaire pour l'**Organisation Nationale de Lutte contre le Faux-Monnayage (ONFM)**, en conformité avec les préconisations de l'agence européenne EMV.

CLASSIFICATION BINAIRE

SCIKIT-LEARN

PYTHON

FASTAPI



Contexte & Objectif

Mission

Développer une application de machine learning permettant aux équipes de l'ONFM de **prédire la nature d'un billet (vrai ou faux)** après scan de ses dimensions physiques. La prédiction repose exclusivement sur 6 mesures géométriques du billet, sans analyse visuelle ou chimique.

Convention cible : faux_billet = 0 (authentique) · faux_billet = 1 (faux).

Périmètre du projet

- Analyse exploratoire et qualité des données
- Comparaison de 4 algorithmes : K-means, Régression logistique, KNN, Random Forest
- Sélection rigoureuse d'un modèle final via validation croisée
- Intégration du pipeline dans une API FastAPI déployable en production

Fichiers

Fichier	Rôle
p12_da_maj+(1).ipynb	Notebook principal
billets.csv, Billets_production.csv	Datasets fournis
model_oncfm.joblib	Modèle final avec preprocessing

1. Vue globale du projet



Le notebook sert à explorer les données, entraîner les modèles et exporter le pipeline final. L'API charge ensuite ce modèle exporté pour produire des prédictions à partir de nouvelles mesures de billets.

Structure Générale du Pipeline

01

Données

Chargement, exploration (EDA)

03

Modélisation

Entraînement RL, KNN, Random Forest, cross-validation multi-modèles, optimisation des hyperparamètres avec `GridSearchCV`.

02

Préparation

Sélection de X et y , séparation train/test 80/20 avec `stratify=y`, encodage et normalisation.

04

Export

Évaluation finale sur le jeu de test, décision qualité, export du pipeline avec `joblib`.



Données & Qualité

1500

Billets scannés

1 000 vrais · 500 faux

7

Variables

diagonal, height_left, height_right, margin_low, margin_up, length + cible

37

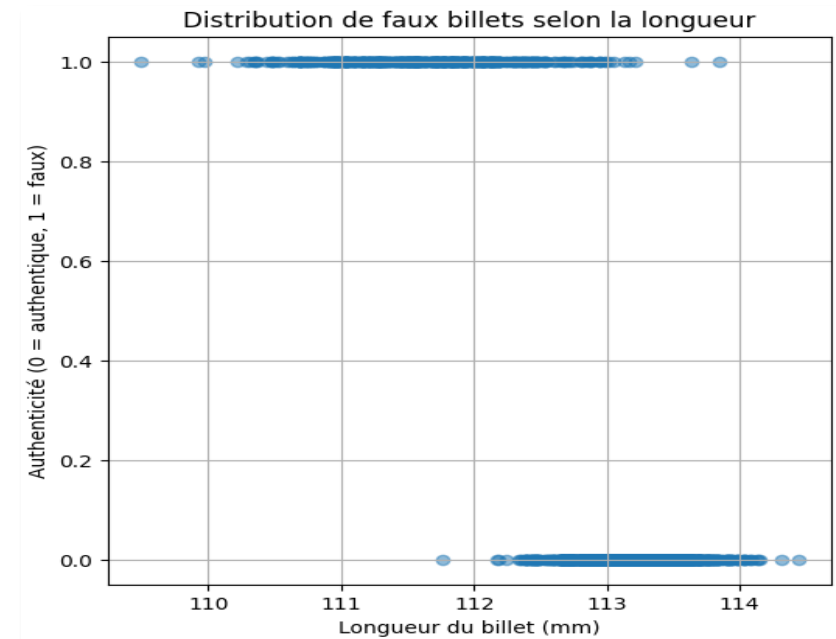
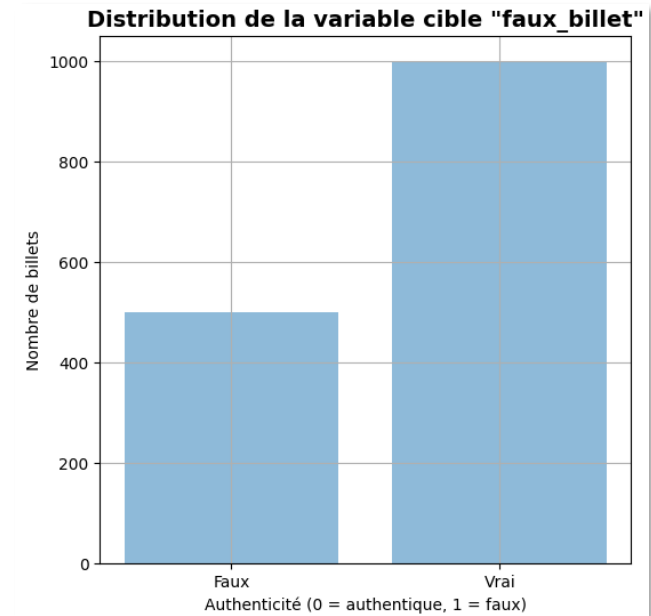
Valeurs manquantes

Uniquement sur `margin_low`

0

Doublons

Jeu de données propre



Analyse Exploratoire & Préparation des Données

Valeurs manquantes

`margin_low` présente **37 valeurs manquantes**. Décision : **imputation par la médiane** intégrée dans le pipeline sklearn, ce qui la rend robuste aux valeurs extrêmes et évite tout biais lors des prédictions futures.

Corrélations

`is_genuine` est fortement corrélée à `length`. La cible est encodée en variable binaire `faux_billet` et les données sont **standardisées obligatoirement** avant chaque modèle sensible à l'échelle (KNN, régression logistique).

Outliers

Détection par **IQR, Z-score** et **IsolationForest** (300 arbres, 3% contamination) → **45 outliers** identifiés. Traitement par Z-score absolu pour ne pas dégrader les données légitimes.

Split Train/Test

Split stratifié **80/20** : 1 200 lignes en train, 300 en test. Fit uniquement sur le train pour **éviter toute fuite de données**. Reproductibilité garantie avec `random_state=42`.

Analyse Non Supervisée : K-means

Démarche exploratoire

K-means a été testé comme étape d'**exploration non supervisée** avec $k=2$. L'objectif était d'observer si les billets se regroupent naturellement en deux clusters correspondant à vrais et faux billets — sans que l'algorithme connaisse les étiquettes réelles.

La comparaison des clusters avec les vraies classes a été réalisée *après coup* via une table de contingence. Les numéros de clusters sont arbitraires et peuvent être inversés.

Conclusion & décision

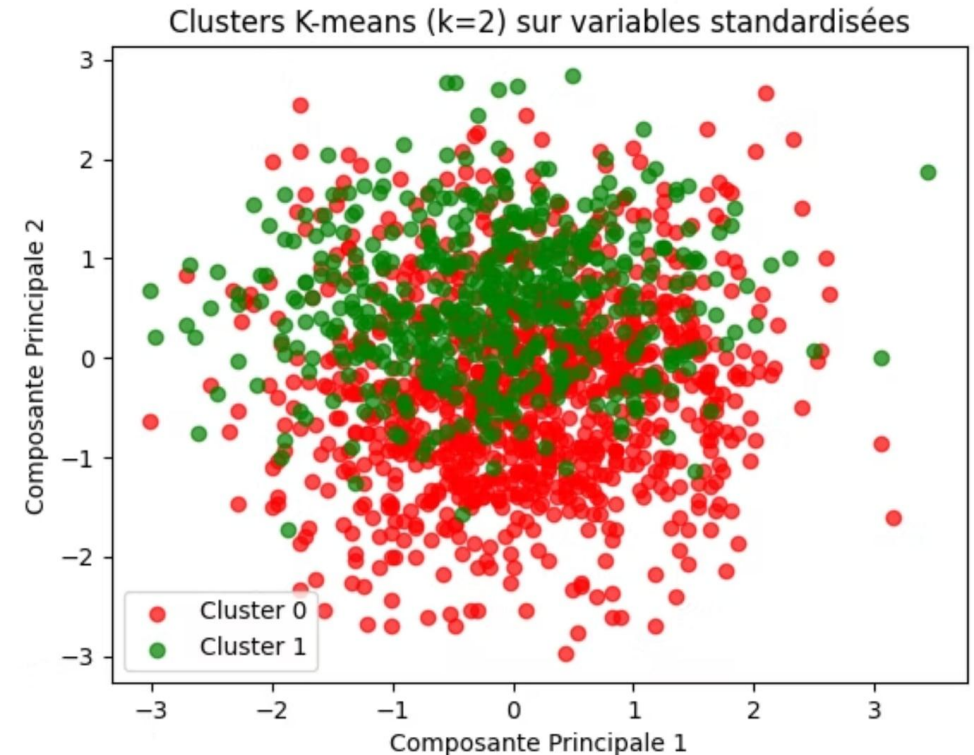
Utile pour

Explorer la structure naturelle des données,
visualiser les séparations en 2D via ACP



Non retenu

K-means ne connaît pas la cible — inadapté
à la prédiction métier de l'ONFM



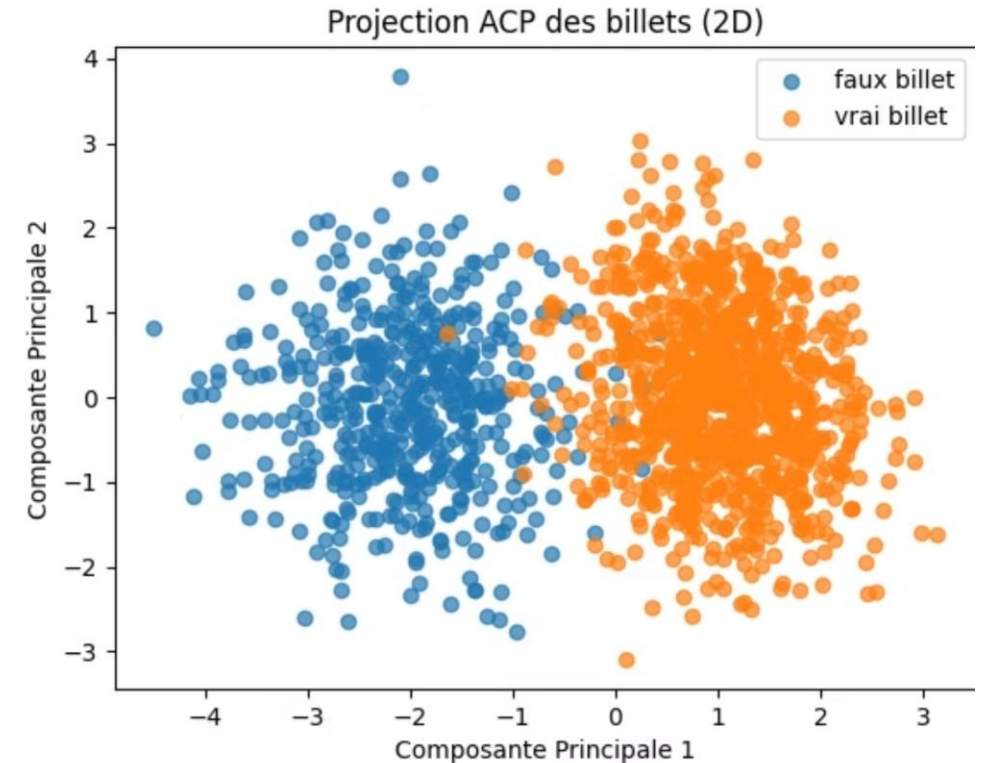
Analyse en Composantes Principales (ACP)

Rôle de l'ACP dans ce projet

L'ACP a été appliquée pour **visualiser la structure des données en 2D**. La projection révèle une séparation partielle mais visible entre faux billets (bleu) et vrais billets (orange) sur la première composante principale, confirmant la pertinence des variables dimensionnelles pour la classification.

Décision finale

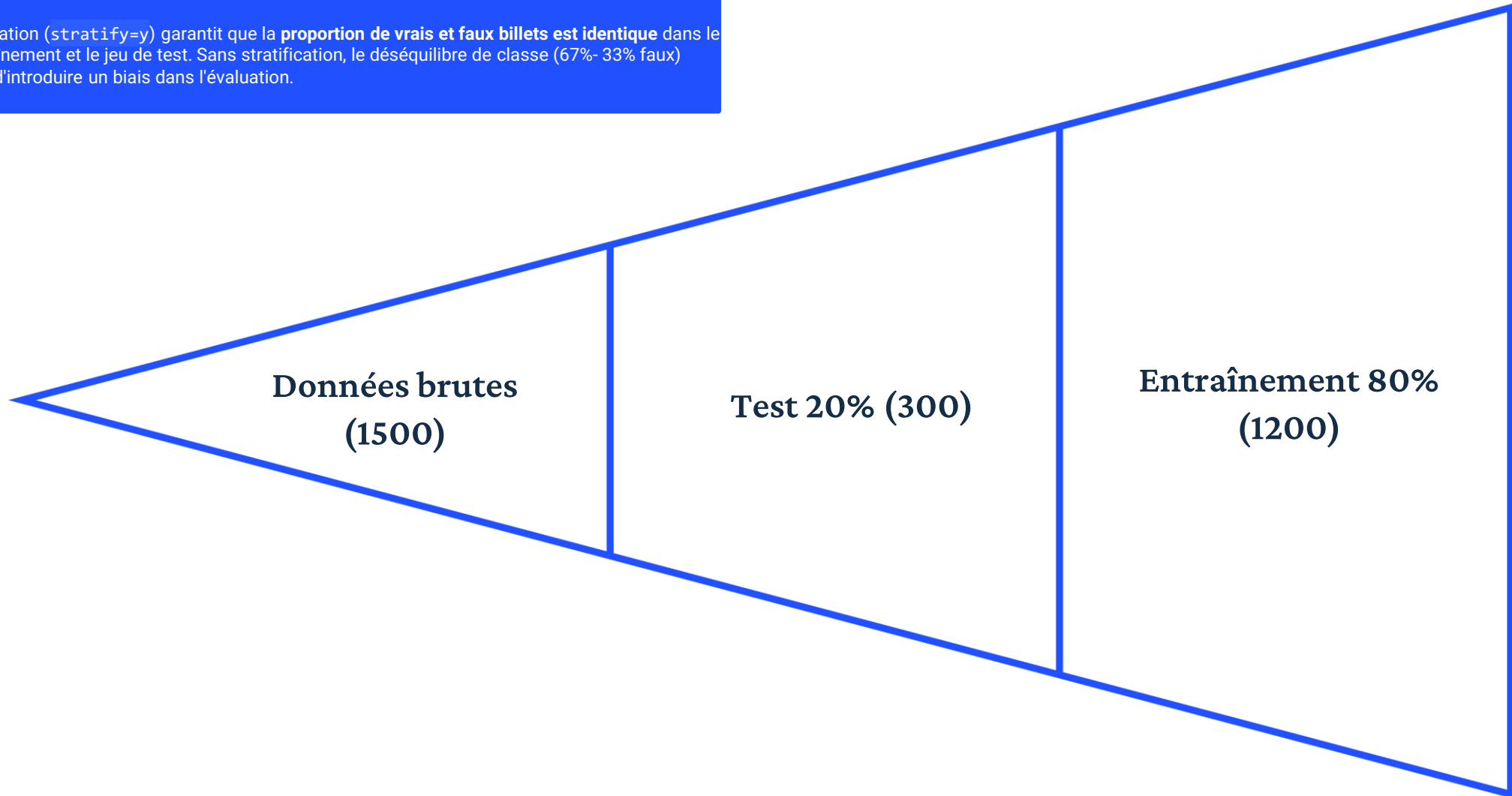
- **Utile** pour comprendre et visualiser la séparabilité des classes
- Plusieurs composantes restent nécessaires pour conserver toute l'information
- **Non retenue** comme étape obligatoire du pipeline final — n'améliore pas significativement la performance des classifieurs testés



Split Train/Test Stratifié

Méthode de stratification

La stratification (`stratify=y`) garantit que la **proportion de vrais et faux billets est identique** dans le jeu d'entraînement et le jeu de test. Sans stratification, le déséquilibre de classe (67%- 33% faux) risquerait d'introduire un biais dans l'évaluation.



Métriques d'Évaluation des Modèles entraînés

Pour un problème de **classification binaire déséquilibré** (2/3 vrais, 1/3 faux billets), l'accuracy seule est insuffisante. Un ensemble de métriques complémentaires est utilisé pour évaluer chaque modèle de façon rigoureuse.



Accuracy

Précision globale du modèle sur l'ensemble des prédictions. Utile en première approche, mais insuffisante seule sur des classes déséquilibrées.



Matrice de confusion

Visualisation détaillée des vrais positifs, faux positifs, vrais négatifs et faux négatifs. Essentielle pour identifier les **faux négatifs** (faux billets classés comme vrais).



Précision • Recall • F1

Métriques essentielles pour les classes déséquilibrées. **Minimiser les faux négatifs est critique** : un faux billet classé comme vrai représente un échec métier direct pour l'ONFM.



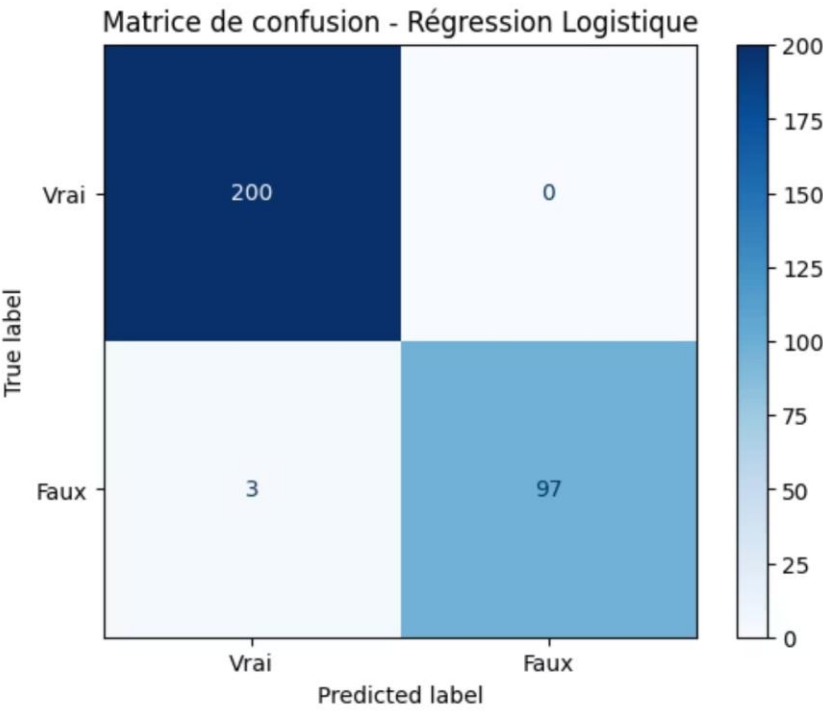
ROC-AUC

Mesure la capacité du modèle à discriminer les deux classes sur **l'ensemble des seuils de décision**. Une valeur proche de 1 indique une excellente séparation.

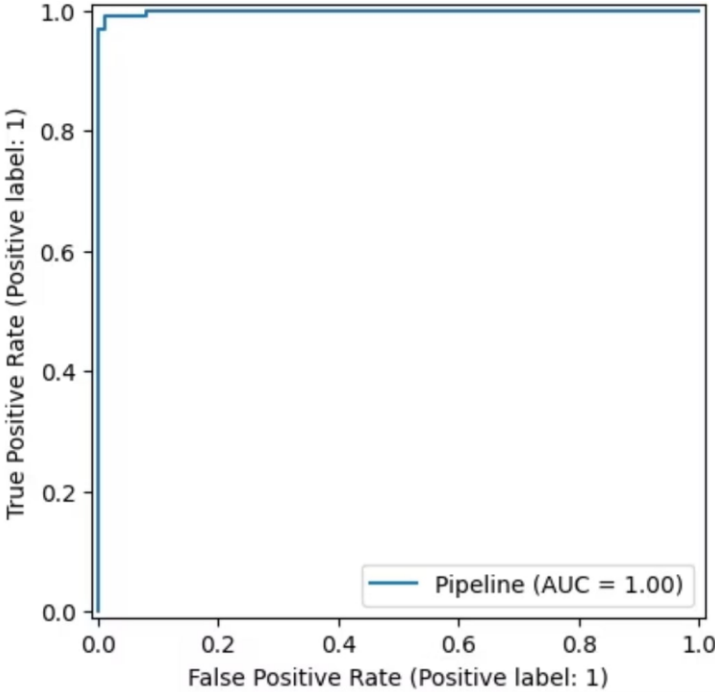
Baseline : Régression Logistique

Pipeline : **imputation** → **standardisation** → **régression logistique**. Ce modèle sert de référence interprétable pour tous les comparatifs ultérieurs.

rank_test_recall	mean_test_recall	mean_test_f1	mean_test_roc_auc	mean_test_accuracy	params
1	0.9974999999999999	0.8839303270571017	0.998296875	0.9125	{'imputation_strategy': 'mean', 'model_C': 0.01, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8898815474337528	0.99875	0.9175000000000001	{'imputation_strategy': 'mean', 'model_C': 0.1, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8951941654302565	0.99865625	0.9216666666666666	{'imputation_strategy': 'mean', 'model_C': 1, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.89700627995538	0.9987031249999999	0.9233333333333335	{'imputation_strategy': 'mean', 'model_C': 1, 'model_fit_intercept': False, 'model_penalty': 'l2', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8960598669776234	0.99865625	0.9225	{'imputation_strategy': 'mean', 'model_C': 10, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8941898342520507	0.998640625	0.9208333333333334	{'imputation_strategy': 'mean', 'model_C': 10, 'model_fit_intercept': False, 'model_penalty': 'l2', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8850507142769487	0.998828125	0.9133333333333333	{'imputation_strategy': 'median', 'model_C': 0.1, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8800228375761044	0.998296875	0.9091666666666667	{'imputation_strategy': 'median', 'model_C': 0.01, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.890854951969706	0.9987343750000001	0.9183333333333333	{'imputation_strategy': 'median', 'model_C': 1, 'model_fit_intercept': False, 'model_penalty': 'l1', 'model_weight_decay': 0.0001}
1	0.9974999999999999	0.8850656152049637	0.998671875	0.9133333333333334	{'imputation_strategy': 'median', 'model_C': 1, 'model_fit_intercept': False, 'model_penalty': 'l2', 'model_weight_decay': 0.0001}



La matrice de confusion montre **0 faux négatif critique** (aucun faux billet classé vrai) et seulement 3 vrais billets rejetés à tort.



La courbe ROC atteint un **AUC = 1,00** sur le jeu de test, démontrant une séparation parfaite des deux classes à l'ensemble des seuils.

✓ Très performante

Accuracy 0,99 · Recall faux 0,97 · ROC-AUC 0,999

✓ Interprétable

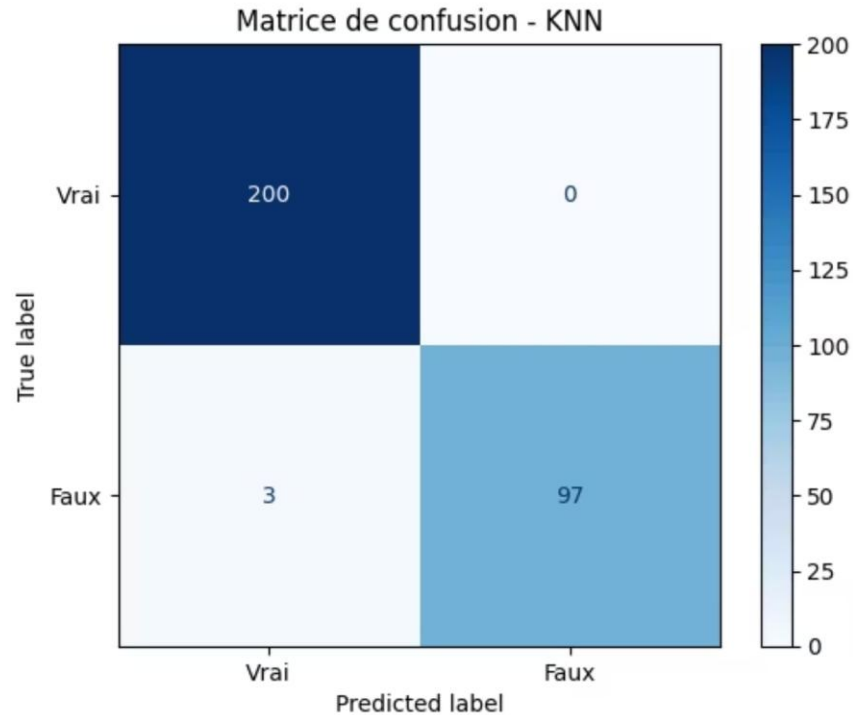
Coefficients lisibles — utile pour audits réglementaires

📌 Référence

Sert de baseline pour tous les comparatifs

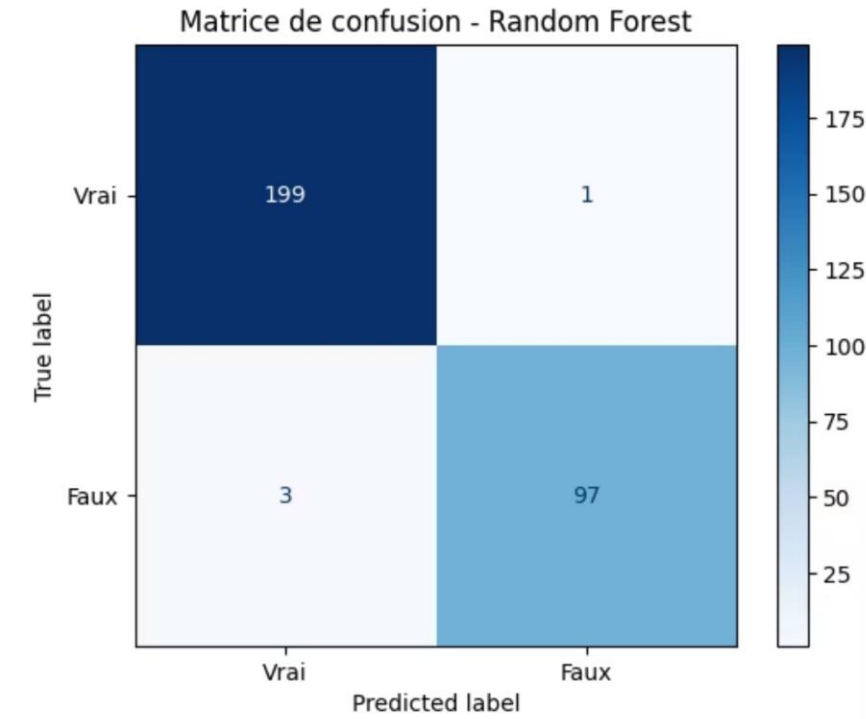
KNN & Random Forest : Comparaison

KNN (K-nearest neighbors)



- Bons résultats sur ce jeu de données
- **Sensible à l'échelle** → standardisation obligatoire
- Performance dépend fortement **du paramètre `n_neighbors`**
- **ROC-AUC légèrement inférieure : 0,984**

Random Forest



- Capture les **relations non linéaires** entre variables
- Robuste aux outliers et aux valeurs manquantes
- Moins sensible à l'échelle que KNN
- **ROC-AUC = 0,999 · Recall faux = 0,98**

❏ Les 3 modèles supervisés sont très performants. Une méthode rigoureuse de sélection par validation croisée est nécessaire pour choisir objectivement.

Benchmark des Algorithmes

Les quatre approches ont été évaluées sur le jeu de test. K-means est présenté à titre exploratoire uniquement (Silhouette score : 0,344 — non retenu pour la prédiction métier).

Modèle	Accuracy	Recall (faux)	F1-score	ROC-AUC
Régression logistique	0,99	0,97	0,98	0,999
KNN (k=5)	0,99	0,97	0,98	0,984
Random Forest 	0,99	0,98	0,98	0,999
K-means (non supervisé)	—	—	—	ARI: 0,929

 Les trois modèles supervisés affichent des performances très proches. Une sélection rigoureuse par validation croisée est nécessaire pour départager.

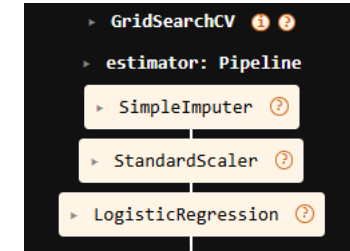
Recherche d'Hyperparamètres & GridSearchCV

Méthode employée

- **GridSearchCV** avec StratifiedKFold (5 folds)
- Scoring multiple : accuracy, precision, recall, F1, ROC-AUC
- **Critère prioritaire : recall** — minimiser les faux billets non détectés
- Recherche exhaustive sur la grille de paramètres pour chaque modèle

Pourquoi cette approche ?

- Comparer les modèles de façon **équitable et reproductible**
- Éviter le choix arbitraire des hyperparamètres
- Obtenir une estimation de performance plus **stable et généralisable**



Meilleurs hyperparamètres (Régression Logistique)

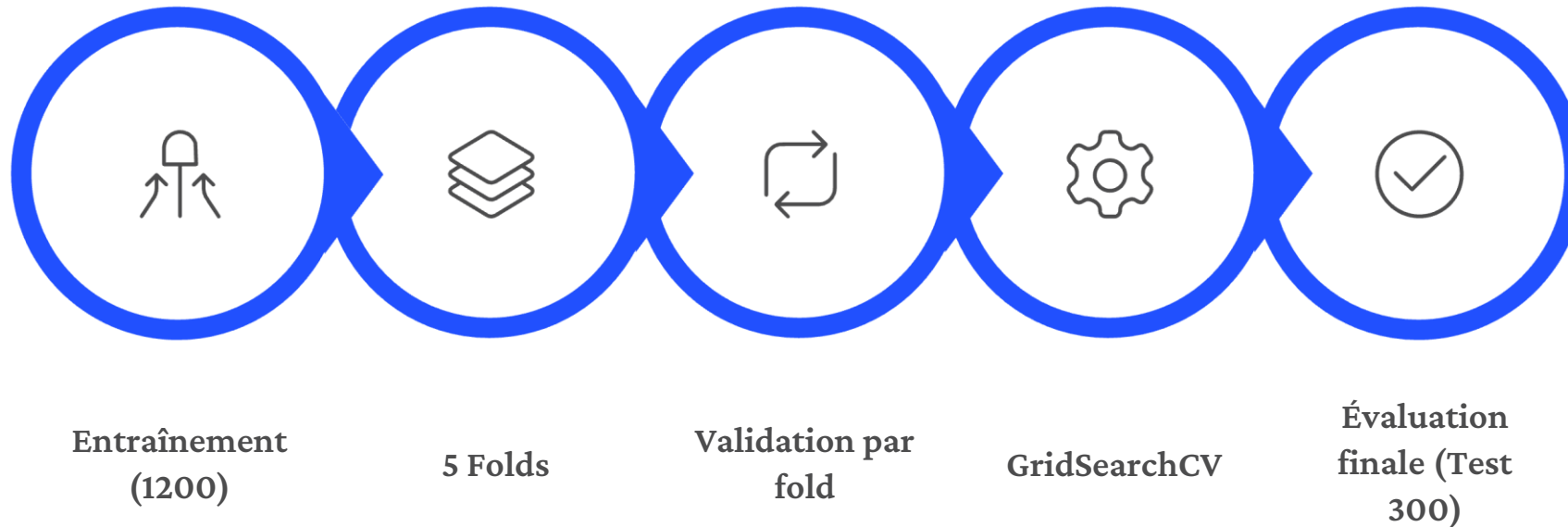
```
strategy: 'mean' · C: 0.01 · fit_intercept: False · penalty:  
'l2' · solver: 'liblinear' · scaler: RobustScaler()
```



Meilleur recall CV : **0,997**

Validation Croisée & Folds

La validation croisée est le mécanisme central qui garantit la fiabilité de la sélection du modèle. Elle permet d'estimer la performance réelle sans consommer les données de test.



Fonctionnement des folds

- Un fold = une portion du jeu d'entraînement (1/5 soit ~240 billets)
- Avec 5 folds, le modèle est **entraîné 5 fois** sur des sous-ensembles différents
- Chaque fold sert **une fois à la validation**, les 4 autres à l'entraînement

Isolation du jeu de test

- La validation croisée se fait **uniquement sur X_train**
- X_test reste à part et n'est jamais vu pendant l'optimisation
- X_test sert **uniquement à l'évaluation finale** — garant d'une estimation non biaisée

Sélection du Meilleur Modèle et Sauvegarde

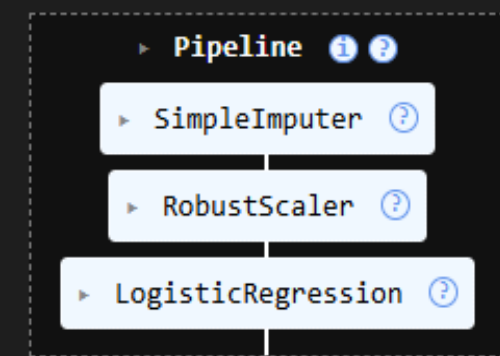
Critères de sélection (par ordre de priorité)

1. **Recall classe faux billet** — critère métier prioritaire de l'ONFM
2. **F1-score** — départage en cas d'égalité sur le recall
3. **ROC-AUC** — discrimination globale des deux classes

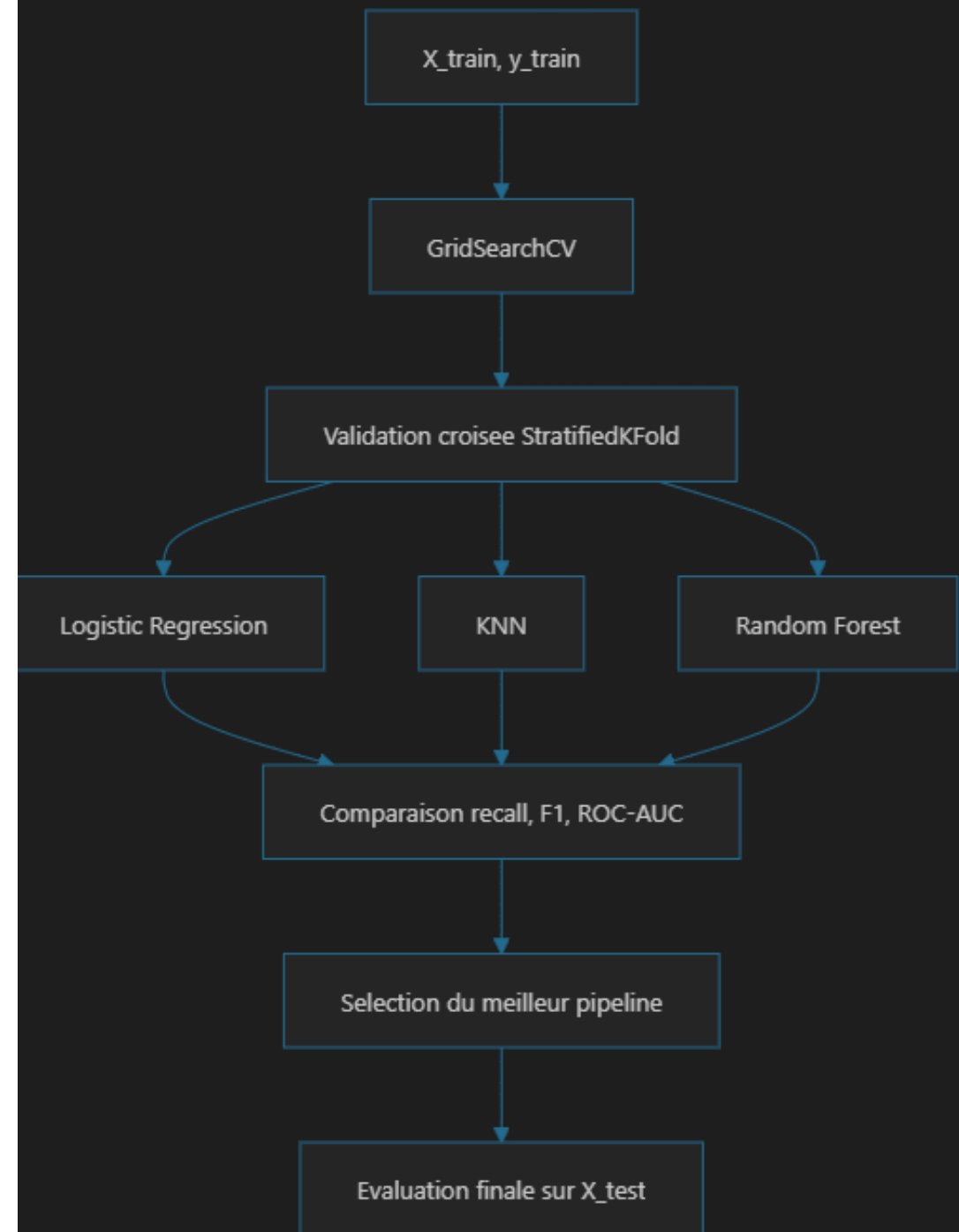
Évaluation finale

Les meilleurs pipelines issus du GridSearchCV sont comparés sur leurs scores de validation croisée, puis le modèle sélectionné est évalué une unique fois sur X_{test} pour obtenir les métriques finales non biaisées.

Pipeline final retenu : LogisticRegression



modele	best recall cv	best f1 cv	best roc_auc cv	best accuracy cv
LogisticRegression	0.9974999999999999	0.8839303270571017	0.998296875	0.9125
RandomForest	0.9875	0.9874674753982255	0.999390625	0.9916666666666666
KNN	0.9825000000000002	0.9861944765445247	0.9953515625	0.9908333333333335



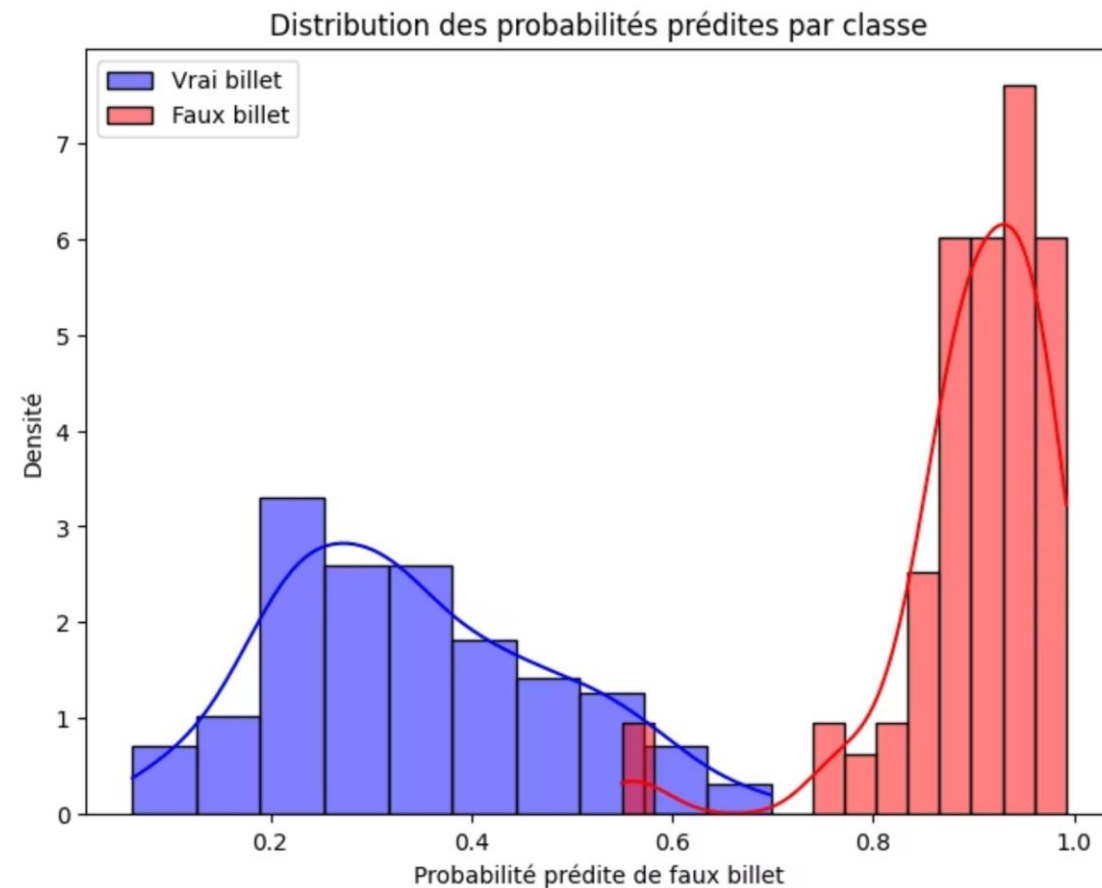
Vérification Overfit & Probabilités

Détection du surapprentissage

```
... Matrice de confusion sur le test :  
[[168  32]  
 [  0 100]]  
  
Rapport de classification sur le test :  
              precision    recall  f1-score   support  
  
   Vrai         1.00        0.84        0.91        200  
   Faux         0.76        1.00        0.86        100  
  
 accuracy              0.89        300  
 macro avg              0.88        300  
 weighted avg           0.92        300
```

Comparaison des scores train / validation / test. L'objectif est de vérifier que le modèle **généralise correctement** et ne mémorise pas les données d'entraînement. Des écarts faibles entre les trois scores valident l'absence d'overfit.

Distribution des probabilités



- **Vrais billets** → probabilité de faux proche de 0
- **Faux billets** → probabilité de faux proche de 1
- Valeurs autour de 0,5 = cas incertains nécessitant vérification manuelle

Export & Inspection du Modèle

Le modèle final est exporté avec `joblib.dump(best_model, 'model_oncfm.joblib')`. Ce fichier contient un **objet Python sérialisé** — pas du code lisible — incluant le preprocessing, les valeurs apprises, l'encodeur et le modèle RL final.

Charger

```
model = joblib.load(
    'modele_oncfm
    .joblib')
```

Inspecter

```
model.named_steps
model.get_params()
```

Prédire

```
model.predict(X_test)
model.predict_proba(
    X_test)[: , 1]
```

Features

```
preprocessor

.get_feature_names_out()
```

Script Minimal d'Utilisation

Chargement du modèle et prédiction unitaire en Python :

```
import joblib
import pandas as pd

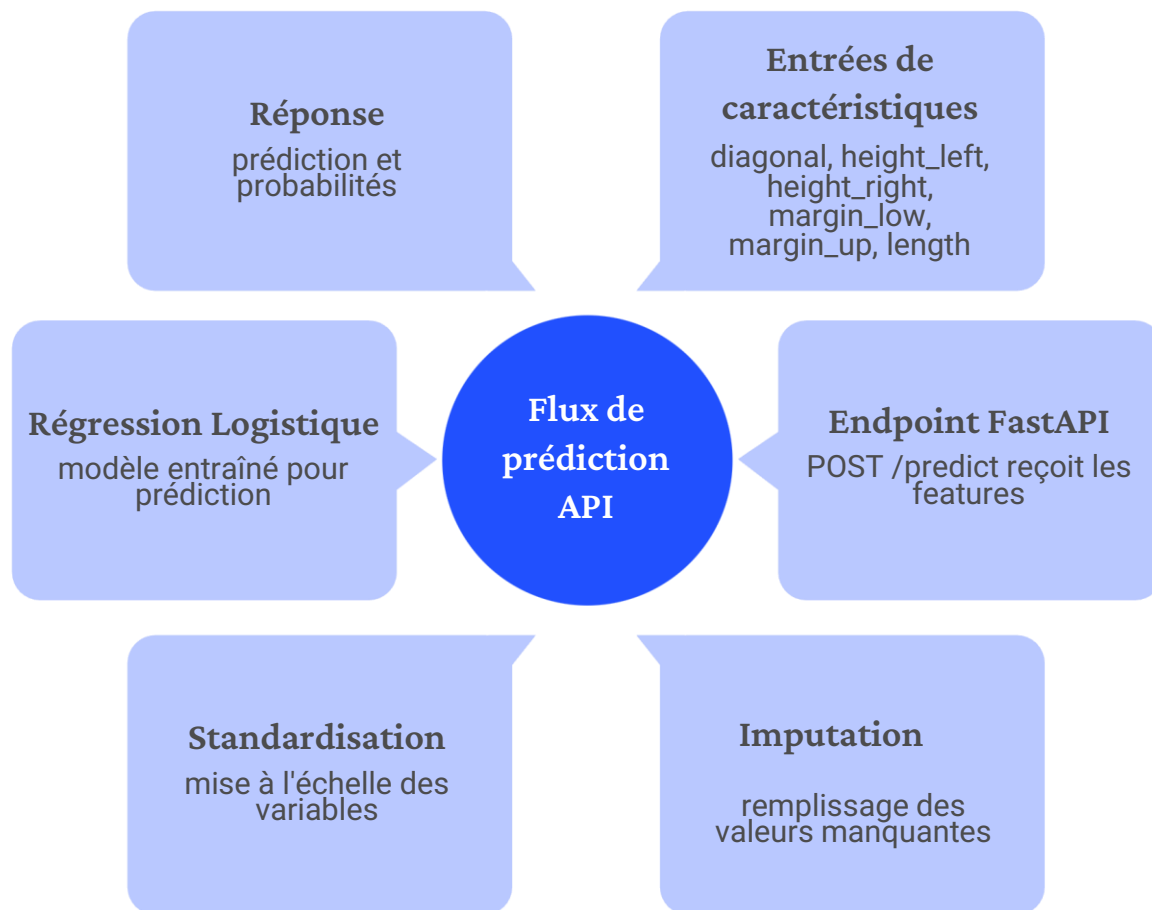
model = joblib.load("model_oncfm.joblib")

sample = pd.DataFrame([{"diagonal": 171.81,
    "height_left": 104.86,
    "height_right": 104.95,
    "margin_low": 4.52,
    "margin_up": 2.89,
    "length": 112.83,
}])

pred = model.predict(sample)[0]
proba_faux = model.predict_proba(sample)[0, 1]
print(pred, round(proba_faux, 3))
```

 Le service contrôle que l'entrée contient bien les 6 features attendues avant toute prédiction.

API FastAPI : Architecture & Endpoints



EndPoints

- GET / – Page d'accueil de l'API
- GET /health – État du service et disponibilité du modèle
- POST /predict – Prédiction unitaire (un billet)
- POST /predict/batch – Prédiction en lot (fichier CSV)

Format de la réponse API

- prediction : "VRAI" ou "FAUX"
- is_genuine : booléen (true/false)
- proba_vrai : probabilité d'être un billet authentique
- proba_faux : probabilité d'être un faux billet

Flux complet : API → Pipeline ML → Prédiction + probabilités



Le point clé est l'utilisation d'un **Pipeline sklearn** : il regroupe preprocessing et modèle dans un seul objet exportable, propre et reproductible.

Conclusion & Limites

Réalisations du projet

→ Pipeline complet

De l'exploration des données jusqu'au déploiement via API
FastAPI fonctionnelle

→ Sélection rigoureuse

4 modèles comparés par validation croisée avec critère métier
(recall) prioritaire

→ Qualité production

Tests automatisés, suivi des erreurs métier, modèle exporté en
`.joblib`

Limites & Perspectives

Limites actuelles

- Dataset limité à 1 500 billets
- Modèle dépend des mesures disponibles
- Surveillance en production nécessaire

Ouvertures possibles

- Réentraînement avec de nouvelles données
- Ajustement du seuil de décision
- Enrichissement avec de nouvelles variables dimensionnelles